CLIP: Campus Listing and Item Platform

IISc Item Discovery and Buyer-Seller Matching Platform

Project Requirements Document

DS252 - Introduction to Cloud Computing
CDS, Indian Institute of Science

Team: Cumulonimbus

1 Problem Statement and Motivation

IISc students currently use informal messaging groups and similar channels to advertise used items, request items, and contact potential buyers or sellers. These channels work for posting information, but chronological messages make it difficult to rediscover, compare, filter, or distinguish older active items from items that are no longer available.

This problem becomes more visible during admissions, graduation, hostel changes, internships, and relocation periods, when many students simultaneously wish to acquire or dispose of furniture, electronics, books, bicycles, appliances, and other reusable items.

CLIP is a private, IISc-focused discovery and matching platform. It provides structured item listings, wanted requests, search and filtering, saved items, buyer–seller conversations, structured offers, and seller-controlled availability.

The platform deliberately **does not process payments or verify physical transactions**. Its responsibility ends at discovery, communication, offer exchange, buyer selection, optional sharing of contact information, and the seller declaring that an item is no longer available.

2 Scope, Exclusions, and Assumptions

In scope: IISc-restricted authentication; temporary lower-trust fresher accounts; sale listings and wanted requests; listing images; search, filters, sorting, and saved listings; private conversations and structured offers; seller selection and availability control; deliberate contact sharing; basic content validation; selected email notifications; and public-cloud deployment with monitoring, recovery, security, and cost evaluation.

The MVP uses adaptive polling for conversation delivery, HTTPS/TLS for network traffic, and end-to-end encryption for private conversation content. Sensitive-purchase flags and an optional seller declaration of proof-of-purchase availability are included.

Out of scope: payment processing or verification, delivery/pickup coordination, ratings, auctions, quantity-aware partial sales, atomic swap transactions, native mobile applications, offline operation, multilingual UI, WhatsApp-based listing creation, automated pricing, comprehensive moderation, and active multi-region serving.

Assumptions: most regular users possess an IISc-managed Microsoft identity; temporary email verification establishes control of a mail ID rather than proof of admission; real traffic is expected to remain relatively small; buyers and sellers complete inspection, payment, pickup, and handover outside the platform. Performance tests will use a reproducibly generated synthetic dataset of approximately **1,000 listings**; this is an evaluation dataset, not a prediction of active production listings.

3 Stakeholders and Usage Scenarios

The principal stakeholders are IISc buyers and sellers, incoming students using restricted temporary accounts, and the project team operating the service.

Representative workflows are:

- A buyer signs in, searches or filters available listings, opens an item, contacts the seller, exchanges messages, and optionally submits an offer.
- A seller creates a listing with images, receives conversations and offers, selects a buyer if desired, and later marks the listing unavailable.
- A user drafts a wanted request and is shown a possible matching sale listing before deciding whether to publish it.
- A fresher without IISc credentials uses a temporary personal email-verified account with restricted buyer-only capabilities until expiry.

4 Functional Requirements

FR-01 Identity and eligibility
The backend shall grant regular access only after validating a signed Microsoft identity token and verifying that the authenticated account identifier belongs to an approved IISc domain. *Validation: authentication integration tests with valid, expired, incorrectly signed, and non-IISc tokens.*

FR-02 Temporary onboarding
The system shall support lower-trust temporary buyer accounts after successful email OTP verification. Such accounts shall expire automatically after 14 days and shall not self-renew. *Validation: account-lifecycle integration test using controlled timestamps.*

FR-03 Temporary-user restrictions
Temporary users may browse, search, save listings, create wanted requests, start conversations, and submit offers, but shall not create sale listings. *Validation: direct API authorization tests for each restricted operation.*

FR-04 Listing creation
An eligible regular user shall be able to create a listing with a title, description, category, condition, asking price (including 0), negotiability, pickup area, availability period, and at least one image. *Validation: API/UI functional tests with valid and invalid field combinations.*

FR-05 Listing ownership and availability
Only the listing owner may edit a listing or modify its authoritative availability state. An unavailable listing shall be hidden from discovery. *Validation: ownership and state-transition API tests.*

FR-06 Browse and pagination
An authenticated eligible user shall be able to browse currently available listings using paginated results. *Validation: functional test on a seeded multi-page dataset.*

FR-07 Search and filtering
Users shall be able to search listing titles and descriptions and filter results by category, price range, condition, negotiability, and listing type. Results shall support sorting by recency or price. *Validation: deterministic search queries over a labeled dataset.*

FR-08 Wanted requests

Eligible users shall be able to create and manage wanted requests containing a title, description, category, and an optional budget. Before submission, the draft title shall be used to show possible active sale matches. *Validation: UI/API test using seeded matching and non-matching listings.*

FR-09 Saved listings

Users shall be able to save and unsave listings. A previously saved listing may remain visible in the user's saved-items view after becoming unavailable. *Validation: state-transition integration test.*

FR-10 Conversations

A buyer shall be able to create at most one private conversation per buyer–listing pair and exchange text messages with the listing owner. Messages shall be stored as ciphertext in the relational database, and clients shall retrieve newly available messages through the normal application API. *Validation: conversation uniqueness, authorization, ordering, and incremental message-retrieval tests.*

FR-11 Structured offers

A buyer shall be able to submit a structured offer containing an amount, an optional note, a timestamp, and a status within the corresponding conversation. *Validation: offer creation, revision/withdrawal, and ownership tests.*

FR-12 Contact sharing

A conversation participant shall be able to deliberately reveal only that participant's selected phone number, email address, or UPI within the relevant conversation. *Validation: privacy and authorization API tests.*

FR-13 Notifications

The system shall asynchronously send an email for defined events, such as a new conversation. After a specified period, the system shall send a reminder for the unread message in that conversation. *Validation: event-generation and forced-email-failure integration tests.*

FR-14 Image handling

Listing images shall be stored in object storage and referenced by authoritative listing records. *Validation: successful upload, rejected upload, missing-object, and partial-failure tests.*

FR-15 Transaction boundary

The platform shall allow sellers to mark items unavailable but shall not represent payment, item quality, collection, delivery, or handover as platform-verified events. *Validation: UI/API inspection and end-to-end workflow test.*

FR-16 End-to-end encryption

The system shall encrypt private messages and decrypt them only on the clients of the conversation participants. Servers shall store and relay ciphertext without plaintext or decryption keys. *Validation: two-client exchange, unauthorized decryption and tampering rejection, key-change handling.*

FR-17 Sensitive purchases

The system shall flag listings whose asking price exceeds a configurable threshold or whose category is mobile phone or laptop. These listings shall offer an optional seller field, "Proof of purchase available?", displayed to buyers as an unverified seller declaration. *Validation: threshold boundary, category, edit/reclassification, and omitted/Yes/No field tests.*

5 Non-Functional Requirements

For performance requirements, the reference workload will be a synthetic database containing at least **1,000 listings and 5,000 users**, created during testing. Measurements will use a load generator such as **k6** and will run for at least five minutes after warm-up. Final experiments will separately report warm and cold-start behavior where applicable.

NFR-01 Browse latency
At a sustained load of **20 requests/s**, at least **95% of successful browse requests shall complete within 1 s**, excluding image-body transfer. *Measurement: k6 client latency plus backend monitoring.*

NFR-02 Search latency
With the reference dataset and **10 search requests/s**, at least **95% of indexed search requests shall complete within 1 s**. *Measurement: load-generator latency and PostgreSQL query metrics.*

NFR-03 Peak API behavior
During a controlled peak of **50 requests/s for five minutes**, the system shall maintain a **successful-request rate of at least 95%**, excluding intentionally rate-limited abusive requests. *Measurement: load-generator status-code and timeout statistics.*

NFR-04 Search usefulness
On a manually labeled set of at least **20 representative search queries**, the system shall achieve **Success@5** $\geq 80\%$, where Success@5 means that at least one human-labeled relevant result occurs among the first five results. *Measurement: offline evaluation script over the fixed query/listing set.*

NFR-05 Active-chat latency
When both conversation participants have the relevant conversation open, at least **95% of messages shall become visible to the receiving participant within 5 seconds of being sent** under normal workload. *Measurement: end-to-end timestamps from sender submission to recipient display.*

NFR-06 Runtime recovery
With at least two healthy serving instances configured, forced loss of one serving runtime shall restore the configured healthy serving capacity within **240 seconds**, while requests during the experiment are measured for failure. *Measurement: orchestrator events, health checks, and load-generator timeline.*

NFR-07 Database recovery
A documented database restore procedure shall successfully restore the latest available managed backup to a usable test database within **30 minutes**. *Measurement: timed recovery exercise followed by integrity/smoke tests.*

NFR-08 Budget
Total cloud expenditure for implementation and evaluation shall remain below 200 USD, and each load experiment shall have a declared maximum duration and resource/concurrency cap before execution. *Measurement: AWS billing/cost tools and experiment resource logs.*

6 System Architecture and Technology Stack

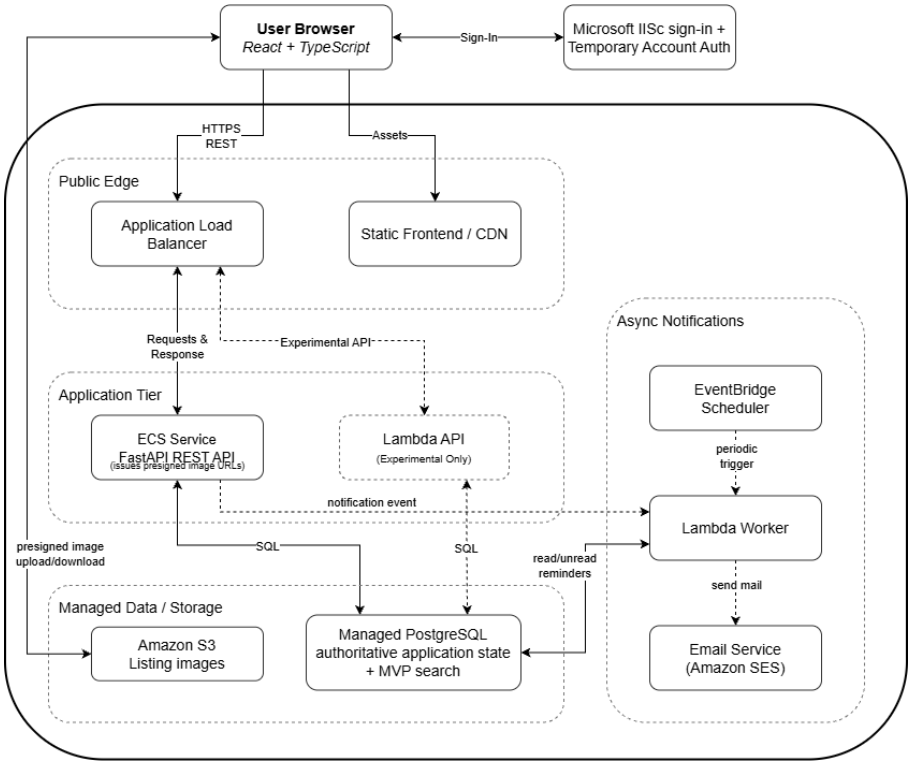
The system uses a responsive React + TypeScript web client, a Python/FastAPI REST backend, managed PostgreSQL for authoritative marketplace state, and S3 for listing images. The normal API is intended to run in ECS containers, while Lambda remains available for asynchronous work and experimental comparison.

The public-facing frontend and approved API ingress are exposed to users, while the database and internal workers remain non-public. Regular authentication is delegated to Microsoft identity; temporary accounts use separately managed credentials and email verification.

Conversation ciphertext uses the same API and PostgreSQL infrastructure as other marketplace states, with lightweight polling and HTTPS/TLS. Client-side end-to-end encryption uses a cryptographic library, authenticated participant keys, and client-held private keys. Key loss cannot be recovered by the server. The MVP supports one active browser device per user.

Unread-chat email reminders are asynchronous. A pending reminder may be kept per recipient/conversation and processed by a scheduled worker; if the conversation is read before the reminder becomes due, no email is sent. Notification delivery does not determine whether the original message succeeds.

PostgreSQL native indexed full-text search is the MVP search implementation. A separate purpose-built search service may be explored later, but PostgreSQL remains authoritative for listing state, ownership, availability, and authorization.



Technology Choices and Alternatives

AWS is selected over a hybrid or multi-cloud deployment because the project does not require institutional infrastructure or cross-cloud portability, while AWS provides the managed compute, database, storage, identity-adjacent, monitoring, and billing capabilities.

React + TypeScript is preferred over separate native mobile applications because one

responsive web client is sufficient for desktop and mobile use. **FastAPI REST** is used because the application consists of conventional resources and actions and does not require GraphQL complexity.

Managed PostgreSQL is preferred over a NoSQL-only architecture because users, listings, conversations, offers, ownership, and availability are relational and require transactions and flexible queries. PostgreSQL full-text search also provides a low-complexity baseline before introducing a separate search system.

S3 stores image objects rather than database BLOBs, so binary storage and delivery remain separate from relational metadata. Short-lived authorized upload URLs are preferred where practical.

ECS is the baseline API runtime because it provides managed container scheduling with less operational complexity than Kubernetes or self-managed virtual machines. **Lambda** remains useful for asynchronous work.

Email is used for asynchronous awareness rather than browser push or native mobile notifications. Managed identity mechanisms are preferred to application-managed institutional passwords.

7 Evaluation Plan

The primary experiment asks:

> *Does offloading bursty, read-heavy marketplace traffic from ECS to Lambda improve peak responsiveness or cost efficiency without degrading normal-load performance?*

The experiment focuses on the read-heavy operations that are central to normal use of the platform, such as browsing available listings, retrieving listing details, and searching/filtering listings. State-changing operations such as listing creation, conversations, offers, and seller actions remain on ECS in both configurations.

Two configurations will be compared:

- **ECS-only:** all benchmarked read-heavy API requests are served by the ECS-hosted FastAPI application.
- **Mixed ECS+Lambda:** selected read-heavy endpoints are served using Lambda, while the remaining application continues to execute on ECS. PostgreSQL remains the authoritative datastore in both configurations.

The two configurations will use the same application logic wherever practical, the same synthetic PostgreSQL dataset of approximately 1,000 listings, the same AWS region, and the same request payload and query distribution. The exact set of endpoints offloaded to Lambda will be fixed before measured runs begin and reported with the final results.

Metrics

For each workload and configuration, the experiment will record:

- p50, p95, and p99 end-to-end request latency;
- *Offered throughput* (number of requests per second that k6 attempts to start) vs *Successful throughput* (number of benchmark requests per second that complete before the client timeout, and return the endpoint's expected HTTP status);
- request failure, timeout, and throttling rate;

- ECS healthy/running task count and scaling timeline where applicable;
- Lambda concurrency and cold-start behavior where applicable;
- PostgreSQL query latency, connection count, and other indicators of database saturation;
- measured or estimated compute cost for the experiment; and
- cost per fixed amount of successful work, such as cost per 1,000 successful requests.

Responsiveness and cost will be evaluated together rather than forcing both configurations to have exactly equal cost or exactly equal latency. In particular, the results will show whether the mixed configuration provides lower tail latency during bursts, and whether any such improvement is accompanied by higher or lower cost.

Evidence and Interpretation

Each measured workload will be repeated where practical. Cold and warm behavior will be distinguished when Lambda initialization, container startup, or cache state materially affects the result. The team will retain the exact workload definition, resource configuration, application version, experiment duration, raw measurements, and cost calculation method.

The conclusions will be limited to the tested workload and resource configurations. A result will not be interpreted as showing that ECS or Lambda is universally superior. Instead, the experiment will determine whether offloading selected read-heavy routes is beneficial for this application's specific low-normal-burst traffic pattern, while also checking whether the shared PostgreSQL database becomes the limiting factor.

If time permits after the primary experiment, we may perform small exploratory comparisons of PostgreSQL-native search against a purpose-built search service and/or measure chat latency and polling overhead under concurrent conversations. These are optimization studies rather than MVP requirements and therefore have no committed experimental targets in this PRD.

8 Work Plan, Milestones, MVP, and Risks

MVP Definition

The MVP comprises IISc authentication and restricted temporary accounts; listing CRUD with images and seller-controlled availability; browse, database-native search, filters, wanted requests, and saved listings; private conversations, structured offers, seller selection, contact sharing, and essential email notifications, and AWS deployment with basic monitoring, database recovery, cost control, and completion of the primary ECS/Lambda/mixed experiment.

The MVP includes polling-based chat with mandatory end-to-end encryption (FR-16), HTTPS/TLS, and sensitive-purchase flags with optional proof-of-purchase declarations (FR-17). Dedicated search infrastructure, advanced moderation, richer matching, distributed tracing, and full Infrastructure-as-Code are stretch goals.

Milestones

- **Week 1 (29 Sep–7 Oct) – Foundation:** agree API/schema contracts, deploy a minimal application, integrate identity providers, and set billing limits.
- **Week 2 (8–14 Oct) – Discovery:** complete regular/temporary account rules, listing CRUD and availability, S3 uploads, browse/search/filter/sort, wanted requests, saved listings.

Integrate and test the discovery workflow; continue the encrypted chat implementation in parallel.

- **Week 3 (15–21 Oct) – Interaction:** integrate encrypted conversations, structured offers, seller selection, contact sharing, and asynchronous email notifications.
- **Week 4 (22–28 Oct) – Hardening:** fix integration defects; verify authorization, monitoring, backup/restore, runtime recovery, and NFRs. Freeze benchmark routes, synthetic data, workloads, and cost caps; complete ECS-only and mixed ECS+Lambda pilot runs.
- **Week 5 (29 Oct–4 Nov) – Evaluation and release:** complete measured runs and analysis; reserve the last few days for failed-run retries, regression fixes, final report, demonstration, and viva evidence.
- **5 November – Submission:** final smoke check and submit code, report, results, and demonstration material.

Stretch work starts only after the required functionality, evaluation, and submission material are complete. If time slips, drop stretch work and optional comparisons, preserving mandatory encryption and the final testing buffer.

Principal Risks

- **Cloud cost overrun:** use billing alerts, bounded experiments, small development resources, and stop/delete unused infrastructure.
- **Unfair compute comparison:** use equivalent benchmark work, workloads, datasets, and success criteria across ECS and Lambda, and document unavoidable differences.
- **Database bottleneck:** monitor database behavior so the compute experiment does not merely measure a shared database limit.
- **External-service delays:** integrate authentication and other external providers early and retain controlled test paths for development.
- **Scope growth:** complete the defined MVP before dedicated search, chat optimizations, moderation, or other stretch features.
- **Encryption integration:** validate library and key lifecycle. Key loss and device limits must be disclosed.
- **Cross-service failures:** keep PostgreSQL authoritative and make image and notification failures observable and safely retryable or recoverable.

References

[1] Amazon Web Services, *Amazon Elastic Container Service Documentation*, *AWS Lambda Documentation*, *Amazon Simple Storage Service Documentation*, AWS Documentation.

[2] PostgreSQL Global Development Group, *PostgreSQL Documentation: Full Text Search*.

[3] FastAPI Project, *FastAPI Documentation*.

[4] Grafana Labs, *k6 Documentation*, load-testing tool considered for reproducible workload generation.

[5] OpenAI, *ChatGPT*, used during PRD preparation for scope stress-testing, surfacing candidate non-functional requirements, restructuring team notes into traceable requirements, and reviewing the proposed architecture and experimental design, accessed September 2026.